ATHERA TREASURY

Core Architecture Blueprint

NEAR Hackathon — Autonomous Business Track
A realistic architecture for a constrained, multi-agent treasury service

Product: Constrained AI-native cross-chain treasury management service for DAOs.
Core idea: Humans set policy once. Agents execute inside that policy. All value movement uses NEAR Intents.

1. Executive Recommendation

Build Athera Treasury as a policy-bound multi-agent system. Humans define constraints once in a versioned configuration. Specialized agents then propose and execute treasury operations, client-related actions, procurement, and settlement exclusively inside those constraints. All cross-chain value movement is expressed as NEAR Intents. Agents never manage bridges or routing directly.

> **Core principle:** Prove one complete, verifiable policy-checked loop (constraint validation → treasury action → NEAR Intent settlement → fee collection → revenue split) before expanding agent roles or adding complex strategies.

2. Scope: What Ships in the MVP

Must ship

• Policy engine that loads and enforces rules from a versioned config.yaml.

• At least one specialized agent (Treasury) that proposes actions only after a policy check.

• NEAR Intent creation and submission for cross-chain movement after policy validation.

• Public ledger of every policy check, intent, cost, fee, and payout.

• Kill switch that immediately stops further agent activity.

• Demo mode that completes one full end-to-end loop on testnet in under two minutes.

• Clear documentation of the custody model, security assumptions, and known limitations.

Must demonstrate

• Exactly where policy is enforced (before any signing or intent submission).

• How a NEAR Intent is constructed, submitted, and tracked to settlement.

• How a reviewer can independently verify a settlement and the associated ledger entry.

• Agent account model and key handling (no full-access keys stored in plaintext).

Defer until after the MVP

• Full multi-agent swarm with concurrent Sales, Procurement, and Accounting agents (can be sequential or stubbed).

• Complex multi-protocol yield optimization or advanced risk models beyond hard limits.

• Production-grade client onboarding interface.

• Sophisticated reporting dashboards beyond the public ledger.

3. Technical Architecture

The system is organized so that policy sits in the critical path of every material action. No agent can move value or incur cost without a successful policy check.

Recommended flow

1. Policy Engine loads the current config.yaml and exposes a validation interface.

2. Treasury Agent monitors positions or receives an instruction and proposes an action.

3. Policy Engine validates the proposal against hard limits (movement size, approved assets, drawdown, etc.).

4. If valid, the Settlement component constructs and submits a NEAR Intent.

5. The intent is tracked to completion and the result is recorded.

6. Accounting logic records any fees and triggers revenue-split intents according to policy.

7. All steps are appended to the public ledger.

8. A kill switch can interrupt execution at any stage.

Recommended stack

Layer	Choice	Reason
Agent Runtime	TypeScript + NEAR agent patterns	Fits NEAR ecosystem and OutLayer custody options
Policy	Versioned config.yaml + validation layer	Human-readable, version-controllable, enforceable
Custody	OutLayer agent custody or scoped function-call keys	Avoids storing full-access keys in plaintext
Settlement	NEAR Intents	Agents declare outcomes; solvers handle cross-chain routing
Logging	Public ledger (events or indexed page)	Independent auditability for reviewers
Demo	Testnet script	Fast, repeatable end-to-end proof

4. Critical Technical Proof (Days 1–2)

Validate the following before investing in a broader agent surface. These are the non-negotiable technical risks.

• An agent account can be created and controlled via OutLayer or a scoped function-call access key.

• A policy check can block an invalid action before any intent is created or any signing occurs.

• A valid action can be turned into a NEAR Intent and tracked through to settlement.

• The result of a settlement can be logged and independently verified by a reviewer.

• The kill switch stops further execution when activated.

> If any of these cannot be demonstrated cleanly, reduce scope rather than paper over the gap.

5. Agent Boundaries (MVP)

Agents are narrowly scoped. No single agent holds broad authority over policy, signing, and strategy.

Agent	Allowed	Forbidden
Treasury	Propose rebalance or movement within current policy	Sign transactions, submit intents, or modify policy
Settlement	Create and submit NEAR Intents after a successful policy check	Decide strategy or alter limits
Accounting	Record events and trigger revenue-split intents per policy	Move client capital outside defined fee rules

Sales and Procurement agents can be added later. In the MVP they may be represented as sequential steps or stubs that still pass through the same policy checks.

5b. Swarm-for-Hire under Policy

Athera can safely participate in or host a constrained Swarm-for-Hire model (as described in the NEAR hackathon ideas from Illia / root.near). External or specialized agents may be temporarily added to perform narrow tasks (data analysis, risk research, strategy proposals), but they never receive direct signing authority.

How it works inside Athera

• A job or bid is posted on-chain or via the Procurement Agent, describing the required work and the maximum NEAR that may be spent on inference.

• External agents can join by funding their own inference budget. They receive only the task description, relevant context, and a scoped prompt.

• Any proposal or output from a swarm agent is treated as untrusted input. It must still pass through the Policy Engine before it can become an intent.

• Payment to swarm agents (for inference or completed work) is itself a NEAR Intent that is subject to the same daily spend limits and approved-counterparty rules.

• The kill switch and policy engine remain the final authority. A swarm agent cannot bypass them.

> This keeps the Swarm-for-Hire idea fully compatible with Athera’s core constraint model: humans set the rules; agents (including temporary swarm members) only propose; policy and Settlement enforce.

6. Minimum Enforced Rules (config.yaml)

These are hard constraints. Agents cannot override them. The policy engine rejects any action that violates them.

• Maximum inference spend per day

• Maximum capital movement per transaction and per day

• Approved chains, assets, and protocols

• Maximum drawdown / simple risk parameters

• Minimum fee and pricing rules

• Revenue split percentages

• Approved counterparties and vendors

• Kill switch flag

7. Observability and Demo Mode

Every material event (policy check, proposed action, intent submission, settlement result, fee, and payout) is appended to a public ledger. A reviewer must be able to correlate a ledger entry with an on-chain or Intent settlement without relying on private logs.

Demo mode runs one complete cycle on testnet: policy validation → intent creation and submission → settlement confirmation → fee collection and revenue split → ledger entry. The entire loop must complete in under two minutes and be repeatable.

8. Security and Custody

• No full-access keys are stored in plaintext.

• Prefer OutLayer agent custody or scoped function-call access keys for agent accounts.

• Policy validation occurs before any signing or intent submission. Invalid proposals never reach a key.

• Document all security assumptions, what is protected, and what remains out of scope for the MVP.

• The kill switch is a first-class control that can halt further agent activity.

9. Success Criterion

> One real, independently verifiable loop that respects human-defined constraints, settles via NEAR Intents, logs every material step to a public ledger, and can be halted by the kill switch. The repository must be reproducible by a reviewer, and all claims in the README and demo must match what actually executes.

This blueprint prioritizes a single, honest, policy-enforced proof over a broad but unverifiable surface. Expand only after the core loop is working and independently checkable.

— End of Blueprint —